

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«Процедуры, функции, триггеры в PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»
Вариант свой. БД «Passenger»**

Обучающийся Дидык Глеб Олегович
Факультет прикладной информатики
Группа K3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

1. Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2. Оборудование и программное обеспечение

Оборудование: персональный компьютер.

Программное обеспечение: СУБД PostgreSQL, pgAdmin 4, база данных Passenger.

3. Практическое задание

1. Создать 3 процедуры для индивидуальной базы данных.
2. Создать оригинальные триггеры для индивидуальной базы данных.
3. Проверить работу процедур и триггеров на корректных и некорректных данных.
4. Подготовить скрипты разработанных объектов и подтверждающие скриншоты работы.

4. Исходная база данных

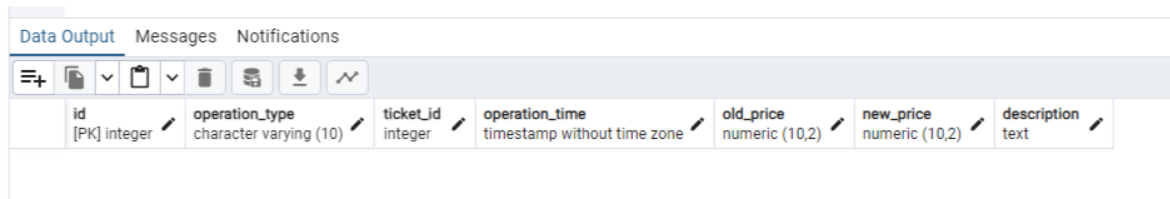
В качестве исходной используется база данных Passenger, созданная в предыдущих лабораторных работах. Рабочая схема базы данных имеет имя passenger_schema. В базе используются таблицы passport, passenger, station, ticket_office, train, carriage, train_carriage, route, route_station, trip и ticket. Также в схеме присутствуют представления, созданные в лабораторной работе №4.


```

id SERIAL PRIMARY KEY,
operation_type VARCHAR(10) NOT NULL,
ticket_id INTEGER,
operation_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
old_price NUMERIC(10,2),
new_price NUMERIC(10,2),
description TEXT
);

```

```
SELECT * FROM ticket_audit_log;
```



id	operation_type	ticket_id	operation_time	old_price	new_price	description
[PK] integer	character varying (10)	integer	timestamp without time zone	numeric (10,2)	numeric (10,2)	text

Рисунок 2 – Создание таблицы ticket_audit_log для логирования действий с билетами

5.2. Создание и проверка процедур

5.2.1. Процедура proc_add_ticket

Процедура proc_add_ticket добавляет новый билет. Пользователь передает понятные текстовые значения: фамилию пассажира, номер кассы, номер маршрута, номер поезда, номер вагона, города отправления и прибытия, стоимость билета и номер места. Внутри процедуры необходимые идентификаторы находятся с помощью SELECT-запросов.

```

CREATE OR REPLACE PROCEDURE proc_add_ticket(
    p_passenger_last_name VARCHAR,
    p_ticket_office_number VARCHAR,
    p_route_number VARCHAR,
    p_train_number VARCHAR,
    p_carriage_number VARCHAR,
    p_departure_city VARCHAR,
    p_arrival_city VARCHAR,
    p_ticket_price NUMERIC,
    p_seat_number INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_station_id INTEGER;
    v_ticket_office_id INTEGER;
    v_passenger_id INTEGER;
    v_trip_id INTEGER;
    v_carriage_id INTEGER;
    v_departure_station_id INTEGER;
    v_arrival_station_id INTEGER;
BEGIN
    SELECT id INTO v_station_id FROM station WHERE city = p_departure_city LIMIT 1;
    SELECT id INTO v_ticket_office_id FROM ticket_office WHERE number = p_ticket_office_number LIMIT 1;
    SELECT id INTO v_passenger_id FROM passenger WHERE last_name = p_passenger_last_name LIMIT 1;

    SELECT tp.id INTO v_trip_id

```

```

FROM trip tp
JOIN route r ON r.id = tp.id_route
JOIN train tr ON tr.id = tp.id_train
WHERE r.number = p_route_number AND tr.number = p_train_number
ORDER BY tp.id LIMIT 1;

```

```

SELECT id INTO v_carriage_id FROM carriage WHERE number = p_carriage_number LIMIT 1;
SELECT id INTO v_departure_station_id FROM station WHERE city = p_departure_city LIMIT 1;
SELECT id INTO v_arrival_station_id FROM station WHERE city = p_arrival_city LIMIT 1;

```

```

IF v_station_id IS NULL THEN RAISE EXCEPTION 'Departure city was not found: %', p_departure_city; END IF;
IF v_ticket_office_id IS NULL THEN RAISE EXCEPTION 'Ticket office was not found: %', p_ticket_office_number; END
IF;
IF v_passenger_id IS NULL THEN RAISE EXCEPTION 'Passenger was not found: %', p_passenger_last_name; END IF;
IF v_trip_id IS NULL THEN RAISE EXCEPTION 'Trip was not found for route % and train %', p_route_number,
p_train_number; END IF;
IF v_carriage_id IS NULL THEN RAISE EXCEPTION 'Carriage was not found: %', p_carriage_number; END IF;

```

```

INSERT INTO ticket (id_station, id_ticket_office, id_passenger, id_trip, id_carriage,
departure_station, arrival_station, ticket_price, sale_datetime, seat_number)
VALUES (v_station_id, v_ticket_office_id, v_passenger_id, v_trip_id, v_carriage_id,
v_departure_station_id, v_arrival_station_id, p_ticket_price, CURRENT_TIMESTAMP, p_seat_number);
END;
$$;

```

```

121 $$;
Data Output Messages Notifications
ЗАМЕЧАНИЕ: процедура
proc_add_ticket(pg_catalog.varchar,pg_catalog.varchar,pg_catalog.varchar,pg_catalog.varchar,pg_catalog.varchar,pg_catalog.varchar,pg_catalog.varchar,pg_catalog.numeric,pg_catalog.int4)
не существует, проницается
CREATE PROCEDURE
Query returned successfully in 77 msec.

```

Рисунок 3 – Создание процедуры proc_add_ticket для добавления билета

Проверка процедуры выполнена с помощью команды CALL. В результате в таблице ticket появилась новая запись с ценой 125.00 и номером места 33.

```

CALL proc_add_ticket('Smith', 'OF-001', 'R-101', '101A', '01', 'Minsk', 'Moscow', 125.00, 33);
SELECT * FROM ticket ORDER BY id;

```

	id [PK] integer	id_station integer	id_ticket_office integer	id_passenger integer	id_trip integer	id_carriage integer	departure_station integer	arrival_station integer	ticket_price numeric (10,2)	sale_datetime timestamp without time zone	seat_number integer
1	1	1	1	1	1	1	1	1	105.05	2026-05-01 10:15:00	5
2	2	1	2	2	1	2	1	2	79.53	2026-05-01 11:20:00	18
3	3	8	1	3	1	4	8	2	70.40	2026-05-02 09:40:00	7
4	4	1	2	4	2	5	1	3	110.00	2026-05-03 14:05:00	11
5	5	6	5	5	2	8	6	3	80.75	2026-05-04 16:30:00	22
6	6	4	3	6	3	3	4	5	68.20	2026-05-05 13:10:00	31
7	7	1	1	7	3	6	1	5	54.90	2026-05-06 12:00:00	40
8	8	6	5	8	4	7	6	7	59.99	2026-05-07 17:45:00	12
9	9	1	2	9	5	1	1	2	105.60	2026-05-08 10:10:00	16
10	11	1	1	1	1	1	1	2	132.00	2026-05-11 18:06:16.704953	30
11	12	1	1	1	1	1	1	2	125.00	2026-05-28 19:08:35.800551	33

Рисунок 4 – Проверка работы процедуры proc_add_ticket

5.2.2. Процедура proc_increase_ticket_price_by_route

Процедура proc_increase_ticket_price_by_route увеличивает стоимость билетов для указанного маршрута на заданный процент. Если процент меньше или равен нулю, процедура выдает ошибку.

```
CREATE OR REPLACE PROCEDURE proc_increase_ticket_price_by_route(  
    p_route_number VARCHAR,  
    p_percent NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF p_percent <= 0 THEN  
        RAISE EXCEPTION 'Percent must be greater than zero';  
    END IF;  
  
    UPDATE ticket  
    SET ticket_price = ticket_price * (1 + p_percent / 100)  
    WHERE id_trip IN (  
        SELECT tp.id  
        FROM trip tp  
        JOIN route r ON r.id = tp.id_route  
        WHERE r.number = p_route_number  
    );  
END;  
$$;
```

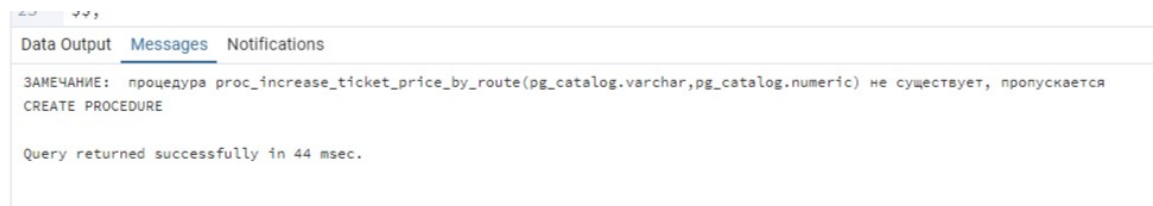


Рисунок 5 – Создание процедуры proc_increase_ticket_price_by_route

Проверка выполнена для маршрута R-101. После вызова процедуры цены билетов по этому маршруту увеличились на 10%.

```
CALL proc_increase_ticket_price_by_route('R-101', 10);
```

```
SELECT t.id, r.number AS route_number, t.ticket_price  
FROM ticket t  
JOIN trip tp ON tp.id = t.id_trip  
JOIN route r ON r.id = tp.id_route  
WHERE r.number = 'R-101'  
ORDER BY t.id;
```

23 ORDER BY t.id;

Data Output Messages Notifications

	id integer	route_number character varying (20)	ticket_price numeric (10,2)
1	1	R-101	115.56
2	2	R-101	87.48
3	3	R-101	77.44
4	9	R-101	116.16
5	11	R-101	145.20
6	12	R-101	137.50

Рисунок 6 – Проверка работы процедуры proc_increase_ticket_price_by_route

5.2.3. Процедура proc_delete_tickets_by_passenger

Процедура proc_delete_tickets_by_passenger удаляет все билеты пассажира по его фамилии. Идентификатор пассажира определяется во вложенном SELECT-запросе.

```
CREATE OR REPLACE PROCEDURE proc_delete_tickets_by_passenger(
    p_passenger_last_name VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM ticket
    WHERE id_passenger IN (
        SELECT id
        FROM passenger
        WHERE last_name = p_passenger_last_name
    );
END;
$$;
```



Рисунок 7 – Создание процедуры proc_delete_tickets_by_passenger

После вызова процедуры для пассажира Smith итоговый SELECT возвращает пустой результат. Это подтверждает, что билеты указанного пассажира были удалены.

```
CALL proc_delete_tickets_by_passenger('Smith');
```

```
SELECT t.id, p.last_name, p.first_name, t.ticket_price
FROM ticket t
JOIN passenger p ON p.id = t.id_passenger
WHERE p.last_name = 'Smith'
ORDER BY t.id;
```

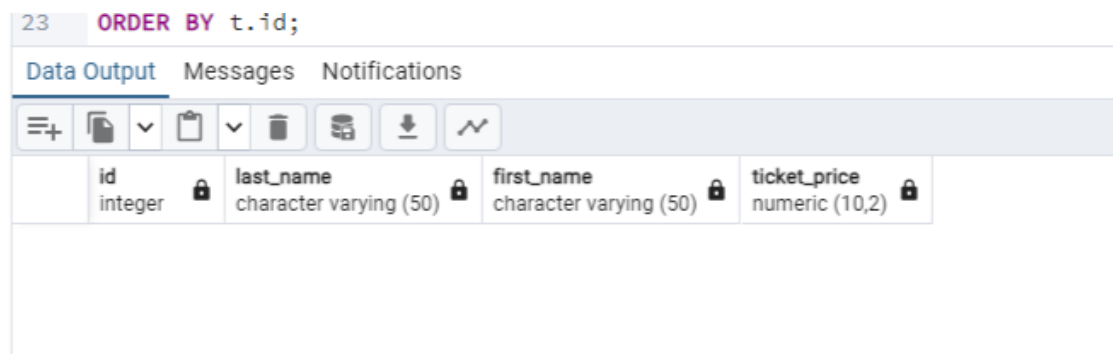


Рисунок 8 – Проверка работы процедуры proc_delete_tickets_by_passenger

5.3. Создание и проверка триггеров

5.3.1. Триггер trg_check_ticket_basic

Триггер trg_check_ticket_basic выполняется перед INSERT или UPDATE в таблице ticket. Он проверяет, что цена билета больше нуля, номер места положительный, станции отправления и прибытия различаются, а дата продажи не находится в будущем.

```
CREATE OR REPLACE FUNCTION fn_check_ticket_basic()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.ticket_price <= 0 THEN
    RAISE EXCEPTION 'Ticket price must be greater than zero';
  END IF;
```



```

IF NEW.seat_number <= 0 THEN
    RAISE EXCEPTION 'Seat number must be greater than zero';
END IF;

IF NEW.departure_station = NEW.arrival_station THEN
    RAISE EXCEPTION 'Departure station and arrival station must be different';
END IF;

IF NEW.sale_datetime > CURRENT_TIMESTAMP THEN
    RAISE EXCEPTION 'Sale datetime cannot be in the future';
END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER trg_check_ticket_basic
BEFORE INSERT OR UPDATE ON ticket
FOR EACH ROW
EXECUTE FUNCTION fn_check_ticket_basic();

```

```

33  FOR EACH ROW
34  EXECUTE FUNCTION fn_check_ticket_basic();

```

Data Output	Messages	Notifications
ЗАМЕЧАНИЕ: триггер "trg_check_ticket_basic" для отношения "ticket" не существует, пропускается		
ЗАМЕЧАНИЕ: функция fn_check_ticket_basic() не существует, пропускается		
CREATE TRIGGER		
Query returned successfully in 38 msec.		

Рисунок 9 – Создание триггера trg_check_ticket_basic для проверки данных билета

Для проверки была выполнена попытка добавить билет с отрицательной стоимостью. Триггер остановил вставку и вывел ошибку.

```

26  );

```

Data Output	Messages	Notifications
ERROR: Ticket price must be greater than zero		
CONTEXT: функция PL/pgSQL fn_check_ticket_basic(), строка 4, оператор RAISE		
ОШИБКА: Ticket price must be greater than zero		
SQL state: P0001		

Рисунок 10 – Проверка работы триггера trg_check_ticket_basic на отрицательной цене билета

5.3.2. Триггер trg_check_ticket_seat

Триггер trg_check_ticket_seat проверяет, что номер места в билете не превышает количество мест в выбранном вагоне. Количество мест берется из таблицы carriage.

```
CREATE OR REPLACE FUNCTION fn_check_ticket_seat()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_seats_count INTEGER;
BEGIN
    SELECT seats_count INTO v_seats_count
    FROM carriage
    WHERE id = NEW.id_carriage;

    IF v_seats_count IS NULL THEN
        RAISE EXCEPTION 'Carriage was not found';
    END IF;

    IF NEW.seat_number > v_seats_count THEN
        RAISE EXCEPTION 'Seat number % exceeds carriage capacity %', NEW.seat_number, v_seats_count;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_check_ticket_seat
BEFORE INSERT OR UPDATE ON ticket
FOR EACH ROW
EXECUTE FUNCTION fn_check_ticket_seat();
```

Проверка выполнена вставкой билета с номером места 999. Так как вместимость вагона равна 36 местам, вставка была отменена.

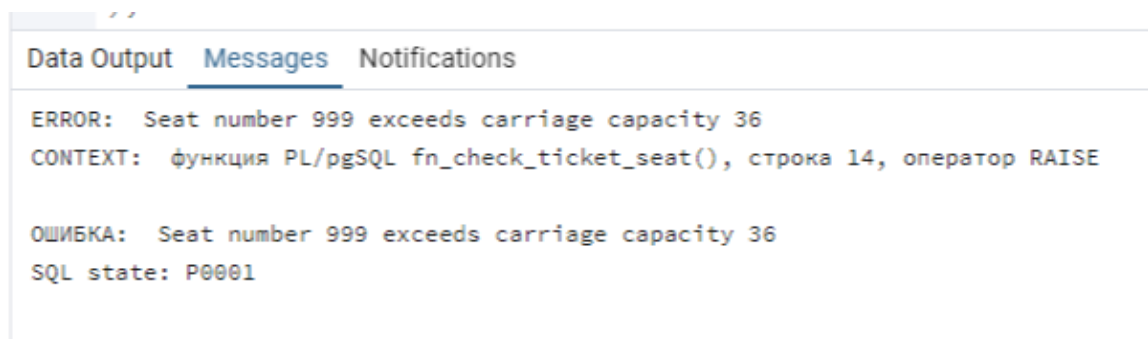


Рисунок 11 – Проверка работы триггера trg_check_ticket_seat на превышении количества мест

5.3.3. Триггер `trg_check_ticket_route_stations`

Триггер `trg_check_ticket_route_stations` проверяет, что станция отправления и станция прибытия входят в маршрут выбранного рейса, а станция отправления идет раньше станции прибытия.

```
CREATE OR REPLACE FUNCTION fn_check_ticket_route_stations()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_route_id INTEGER;
    v_departure_order INTEGER;
    v_arrival_order INTEGER;
BEGIN
    SELECT id_route INTO v_route_id FROM trip WHERE id = NEW.id_trip;

    IF v_route_id IS NULL THEN
        RAISE EXCEPTION 'Trip was not found';
    END IF;

    SELECT station_order INTO v_departure_order
    FROM route_station
    WHERE id_route = v_route_id AND id_station = NEW.departure_station;

    SELECT station_order INTO v_arrival_order
    FROM route_station
    WHERE id_route = v_route_id AND id_station = NEW.arrival_station;

    IF v_departure_order IS NULL THEN RAISE EXCEPTION 'Departure station is not included in the route'; END IF;
    IF v_arrival_order IS NULL THEN RAISE EXCEPTION 'Arrival station is not included in the route'; END IF;
    IF v_departure_order >= v_arrival_order THEN RAISE EXCEPTION 'Departure station must be before arrival station in the route'; END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_check_ticket_route_stations
BEFORE INSERT OR UPDATE ON ticket
FOR EACH ROW
EXECUTE FUNCTION fn_check_ticket_route_stations();
```

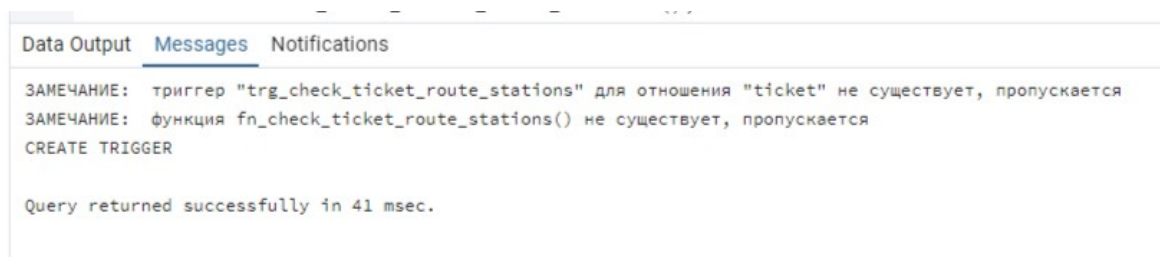


Рисунок 12 – Создание триггера `trg_check_ticket_route_stations` для проверки станций маршрута

Для проверки была выполнена попытка добавить билет, где станция отправления идет после станции прибытия. Триггер вернул ошибку и отменил вставку.

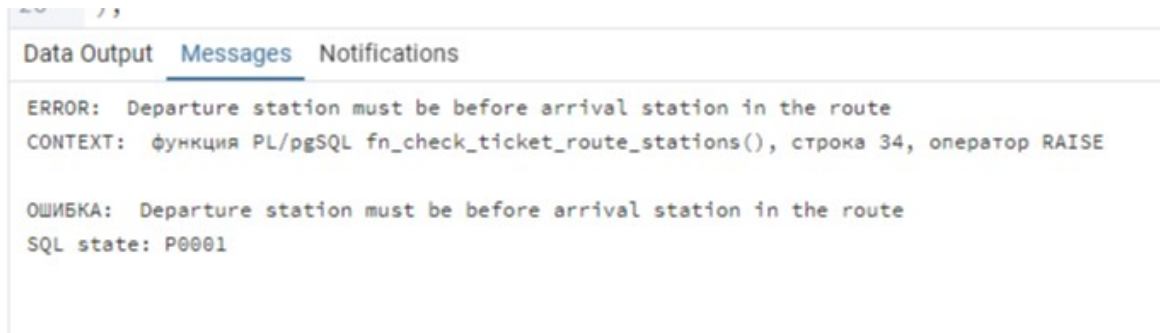


Рисунок 13 – Проверка работы триггера `trg_check_ticket_route_stations` на неправильном порядке станций

5.3.4. Триггер `trg_ticket_audit`

Триггер `trg_ticket_audit` выполняется после `INSERT`, `UPDATE` или `DELETE` в таблице `ticket` и записывает информацию об операции в таблицу `ticket_audit_log`.

```
CREATE OR REPLACE FUNCTION fn_ticket_audit()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO ticket_audit_log (operation_type, ticket_id, new_price, description)
        VALUES ('INSERT', NEW.id, NEW.ticket_price, 'New ticket was added');
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO ticket_audit_log (operation_type, ticket_id, old_price, new_price, description)
        VALUES ('UPDATE', NEW.id, OLD.ticket_price, NEW.ticket_price, 'Ticket was updated');
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO ticket_audit_log (operation_type, ticket_id, old_price, description)
        VALUES ('DELETE', OLD.id, OLD.ticket_price, 'Ticket was deleted');
        RETURN OLD;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_ticket_audit
AFTER INSERT OR UPDATE OR DELETE ON ticket
FOR EACH ROW
EXECUTE FUNCTION fn_ticket_audit();
```

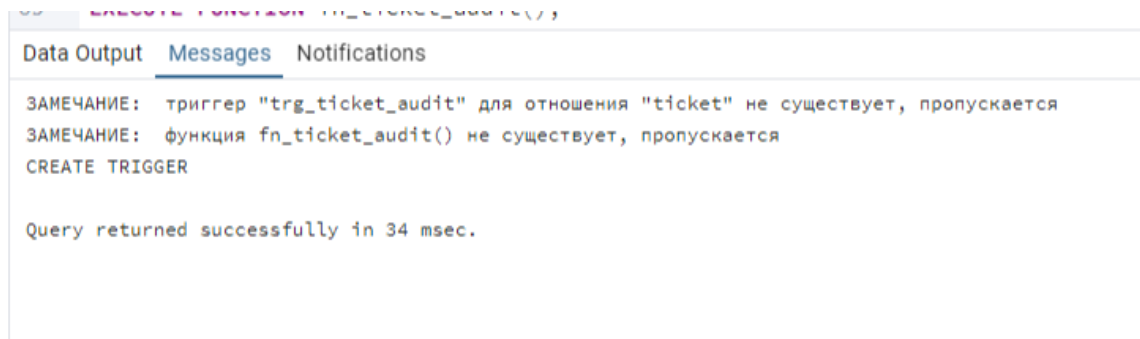


Рисунок 14 – Создание триггера trg_ticket_audit для логирования операций с билетами

Для проверки были выполнены операции UPDATE и DELETE над билетом. В таблице ticket_audit_log автоматически появились записи с типами UPDATE и DELETE.

```
UPDATE ticket SET ticket_price = ticket_price + 5 WHERE id = 2;
DELETE FROM ticket WHERE id = 2;
SELECT * FROM ticket_audit_log ORDER BY id;
```

Data Output Messages Notifications							
	id [PK] integer	operation_type character varying (10)	ticket_id integer	operation_time timestamp without time zone	old_price numeric (10,2)	new_price numeric (10,2)	description text
1	1	UPDATE	2	2026-05-28 19:16:00.710787	87.48	92.48	Ticket was updated
2	2	DELETE	2	2026-05-28 19:16:00.710787	92.48	[null]	Ticket was deleted

Рисунок 15 – Проверка работы триггера trg_ticket_audit и просмотр таблицы ticket_audit_log

5.3.5. Триггер trg_check_trip_datetime

Триггер trg_check_trip_datetime проверяет корректность дат рейса в таблице trip. Дата прибытия должна быть позже даты отправления.

```
CREATE OR REPLACE FUNCTION fn_check_trip_datetime()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.arrival_datetime <= NEW.departure_datetime THEN
        RAISE EXCEPTION 'Arrival datetime must be later than departure datetime';
    END IF;
    RETURN NEW;
END;
$$;
```

```
CREATE TRIGGER trg_check_trip_datetime
BEFORE INSERT OR UPDATE ON trip
FOR EACH ROW
EXECUTE FUNCTION fn_check_trip_datetime();
```

```
CREATE FUNCTION fn_check_trip_datetime()  
--  
Data Output Messages Notifications  
ЗАМЕЧАНИЕ: триггер "trg_check_trip_datetime" для отношения "trip" не существует, пропускается  
ЗАМЕЧАНИЕ: функция fn_check_trip_datetime() не существует, пропускается  
CREATE TRIGGER  
  
Query returned successfully in 35 msec.
```

Рисунок 16 – Создание триггера trg_check_trip_datetime для проверки дат рейса

Проверка выполнена вставкой рейса, у которого время прибытия раньше времени отправления. Триггер остановил вставку и вывел сообщение об ошибке.

```
Data Output Messages Notifications  
ERROR: Arrival datetime must be later than departure datetime  
CONTEXT: функция PL/pgSQL fn_check_trip_datetime(), строка 4, оператор RAISE  
  
ОШИБКА: Arrival datetime must be later than departure datetime  
SQL state: P0001
```

Рисунок 17 – Проверка работы триггера trg_check_trip_datetime на некорректных датах рейса

5.4. Проверка созданных объектов

После создания процедур, функций и триггеров выполнены запросы к системным представлениям PostgreSQL. Они подтверждают наличие созданных объектов в схеме passenger_schema.

```
SELECT trigger_name, event_object_table, action_timing, event_manipulation  
FROM information_schema.triggers  
WHERE trigger_schema = 'passenger_schema'  
ORDER BY event_object_table, trigger_name;
```

```
SELECT proname AS object_name  
FROM pg_proc  
WHERE pronamespace = 'passenger_schema'::regnamespace  
AND (praname LIKE 'proc_%' OR proname LIKE 'fn_%')  
ORDER BY proname;
```

Data Output Messages Notifications				
+ 📄 ▼ 📋 ▼ 🗑️ 📦 ⬇️ 📈				
	trigger_name name	event_object_table name	action_timing character varying	event_manipulation character varying
1	trg_check_ticket_basic	ticket	BEFORE	INSERT
2	trg_check_ticket_basic	ticket	BEFORE	UPDATE
3	trg_check_ticket_route_stations	ticket	BEFORE	UPDATE
4	trg_check_ticket_route_stations	ticket	BEFORE	INSERT
5	trg_check_ticket_seat	ticket	BEFORE	UPDATE
6	trg_check_ticket_seat	ticket	BEFORE	INSERT
7	trg_ticket_audit	ticket	AFTER	UPDATE
8	trg_ticket_audit	ticket	AFTER	DELETE
9	trg_ticket_audit	ticket	AFTER	INSERT
10	trg_check_trip_datetime	trip	BEFORE	INSERT
11	trg_check_trip_datetime	trip	BEFORE	UPDATE

Рисунок 18 – Список созданных триггеров в схеме passenger_schema

Data Output		Messages	Notifications
☰+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈			
	object_name name 🔒		
1	fn_check_ticket_basic		
2	fn_check_ticket_route_stations		
3	fn_check_ticket_seat		
4	fn_check_trip_datetime		
5	fn_ticket_audit		
6	proc_add_ticket		
7	proc_delete_tickets_by_passenger		
8	proc_increase_ticket_price_by_route		

Рисунок 19 – Список созданных процедур и функций в схеме passenger_schema

6. Выводы

В ходе выполнения лабораторной работы были созданы три процедуры для базы данных Passenger: процедура добавления билета, процедура повышения стоимости билетов по маршруту и процедура удаления билетов пассажира. Процедуры были проверены через команды CALL, результаты выполнения подтверждены скриншотами.

Также были разработаны триггерные функции и триггеры для контроля корректности данных и логирования изменений. Триггеры проверяют стоимость билета, номер места, порядок станций в маршруте, корректность дат рейса, а также записывают операции изменения билетов в таблицу ticket_audit_log.

Проверка показала, что некорректные данные блокируются, а действия UPDATE и DELETE автоматически фиксируются в журнале.

Работа позволила закрепить навыки создания процедур, функций, триггеров и проверки их работы в PostgreSQL.

7. Список источников

1. Документация PostgreSQL. CREATE PROCEDURE [Электронный ресурс]. URL: <https://postgrespro.ru/docs/postgresql/11/sql-createprocedure>
2. Документация PostgreSQL. CREATE FUNCTION [Электронный ресурс]. URL: <https://postgrespro.ru/docs/postgresql/11/sql-createfunction>
3. Документация PostgreSQL. Триггеры PL/pgSQL [Электронный ресурс]. URL: <https://postgrespro.ru/docs/postgresql/9.6/plpgsql-trigger>
4. Документация PostgreSQL. SQL Shell psql [Электронный ресурс]. URL: <https://postgrespro.ru/docs/postgresql/12/app-psql>